

Assignment 4

Name: Kaifeng Tang ID: 524030910124

May 28, 2026

Question 1

(a) Since $L = R = 1$, we can know that $i_{j+1} - i_j = 1$ for all $1 < j \leq \ell - 1$. The subsequence becomes a contiguous subsequence. And the problem changes to: Find the Continuous Subsequence Sum.

Define Subproblem: Let $rev(i)$ be the maximum revenue of the (1,1)-step subsequence ended up with a_i .

Subproblem Transfer:

Case 1. Only choose a_i : $rev(i) = a_i$;

Case 2. Besides a_i , Choose some other elements: $rev(i) = rev(i - 1) + a_i$.

$$\Rightarrow rev(i) = \max\{a_i, rev(i - 1) + a_i\}.$$

Objective: $Rev = \max_{i \in (1, n)} \{rev(i)\}$.

Construct DP Algorithm:

Algorithm 1 Find the maximum (1,1)-step subsequence

```
1: Initialize  $rev(1) = a_1$ ;  $Rev = rev(1)$ 
2: for  $i = 2$  to  $n$ : do
3:    $rev(i) = \max\{a_i, rev(i - 1) + a_i\}$ 
4:    $Rev = \max\{Rev, rev(i)\}$ 
5: end for
6: return  $Rev$ 
```

Time Complexity: at most n rounds; compute for $O(1)$ time

$\Rightarrow$ Overall Time: $O(n)$.

Proof of Correctness:

From **Subproblem Transfer** above we prove the correctness of Transfer Function.

Since $rev(i) = \max\{a_i, rev(i - 1) + a_i\}$, we get to know $rev(i)$ actually from $rev(i - 1)$. So from 1 to n is in the topological order.

(b) For any L and R , from $L \leq i_{j+1} - i_j \leq R$ we have $i_{j+1} - R \leq i_j \leq i_{j+1} - L$. Still, we use the DP idea similar to (a).

Define Subproblem: Let $rev(i)$ be the maximum revenue of the **(L,R)**-step subsequence ended up with a_i .

Subproblem Transfer: Two cases just the same as (a), only a condition changes:

$$rev(i) = \max\{a_i, rev(j) + a_i\}, \text{ where } j \text{ satisfies } i - R \leq j \leq i - L$$

if no suitable j in $(i - R, i - L)$, then $rev(i) = a_i$.

Construct DP Algorithm:

Algorithm 2 Find the maximum (L,R)-step subsequence

```

1: for  $i = 1$  to  $n$ : do
2:   Initialize  $best = 0$ 
3:   for  $j = \max(1, i - R)$  to  $i - L$ : do
4:      $best = \max(best, j)$ 
5:   end for
6:    $rev(i) = a_i + best$ 
7: end for
8:  $Rev = \max\{rev(1), rev(2), \dots, rev(n)\}$ 
9: return  $Rev$ 

```

Time Complexity: at most n rounds; check suitable j for $O(n)$ time

$\implies$ Overall Time: $O(n^2)$.

Proof of Correctness: From **Subproblem Transfer** above, we can get

$$rev(i) = \max\{a_i, rev(j) + a_i\} = a_i + \max\{0, rev(j)\}, i - R \leq j \leq i - L$$

Lead in a new variable $best$ to maintain $\max(0, rev(j))$ part, which is Initialized 0. Through checking j in the specific period, we can get update best.

Since $i - R \leq j \leq i - L$, we get $rev(i)$ from previous rev . So the order 1 to n is the topological order.

(c) By algorithm in Solution (b), the algorithm check all the $j \in (i - R, i - L)$ to update $best$, which spends $O(n)$ time. We need to optimize this part.

By the idea of *Potential Prefix*, we don't have to care about all the j when getting a suitable j . Instead, we just need to care about the j that may become the Transfer source in the future.

Let's maintain a queue Q , pushing Potential prefix j in it.

Maintain queue Q: when Q deals with i -th element, consider $j_1 < j_2$:

- j_1 is "older" than j_2
- if $rev(j_1) < rev(j_2)$, j_1 would never be chosen as a transfer source.
- We say j_1 is "out-of-date", and should be kicked out.

Proof of Correctness:

By this rule, when algorithm is in round i -th, Q should add $new = i - L$. Before that, Q should kick out all tails and fronts "out-of-date".

The result is, Q is always index-increasing and rev -increasing. So $Q.front$ must corresponds to the j with largest $rev(j)$.

So the correct Subproblem Transfer: $rev(i) = a_i + \max(0, rev(Q.front))$.

Algorithm 3 Find the maximum (L, R) -step subsequence with Potential Prefix

```
1: Initialize an empty deque  $Q$ 
2: Initialize  $Rev \leftarrow -\infty$ 
3: for  $i = 1$  to  $n$  do
4:    $new \leftarrow i - L$ 
5:   if  $new \geq 1$  then
6:     while  $Q$  is not empty and  $rev(Q.back) \leq rev(new)$  do
7:       PopBack( $Q$ )
8:     end while
9:     PushBack( $Q, new$ )
10:  end if
11:  while  $Q$  is not empty and  $Q.front < i - R$  do
12:    PopFront( $Q$ )
13:  end while
14:  if  $Q$  is empty then
15:     $rev(i) \leftarrow a_i$ 
16:  else
17:     $rev(i) \leftarrow a_i + \max(0, rev(Q.front))$ 
18:  end if
19:   $Rev \leftarrow \max(R, rev(i))$ 
20: end for
21: return  $Rev$ 
```

Time Complexity:

- Every index are pushed/popped by Q at most once;
- Overall Time: $O(n)$.

Question 2

Consider a sequence of continuous words: $a_i, a_{i+1}, a_{i+2}, \dots, a_j$. By observation, the subtree which consists of this sequence must be a Binary-Search Tree too.

Subproblem Definition:

Define $c(i, j)$: The minimum cost of a Binary-Search Tree with $a_i, a_{i+1}, a_{i+2}, \dots, a_j$.

Assume choosing an arbitrary node a_k as root ($i \leq k \leq j$), then a_k 's contribution: $w_k \times \ell_k(T) = w_k$.

Subproblem Transfer:

- cost of left subtree: $c(i, k - 1)$;
- cost of right subtree: $c(k + 1, j)$.

Define weight sum $S(i) := \begin{cases} 0, & i = 0 \\ w_1 + w_2 + \dots + w_i, & o.w. \end{cases}$

Since all nodes' level in both left and right subtree add 1 due to Transfer, all nodes except a_k add 1 onto $\ell_i(T)$, i.e., a_i 's cost plus w_i , except a_k .

So the Transfer Function:

$$c(i, j) = \min_{k \in [i, j]} \{c(i, k-1) + c(k+1, j) + S(j) - S(i-1)\}$$

Topological Order: $c(i, j)$ depends on shorter ones $c(i, k-1)$, $c(k+1, j)$.

So algorithm should compute by the sequence length from small to large (1 to n).

Algorithms:

OptimalBST($w([1, 2, \dots, n])$):

Initialize weight sum $S[i]$: $S[0] = 0$; $S[i] = S[i-1] + w_i$ for $i = 1$ to n

Initialize $c(i, i-1) = 0$, for $i = 1$ to $n+1$

for $length = 1$ to n :

$c(i, j) \leftarrow +\infty$ for all i, j

 for $k = i$ to j :

$cost = c(i, k-1) + c(k+1, j) + S[j] - S[i-1]$

 update $c(i, j) \leftarrow \min\{cost, c(i, j)\}$; Get optimal Root index k for each $[i, j]$.

return $k(i, j)$

BuildTree(i, j):

 Get optimal root of BST(i, j) from *OptimalBST*(w).

 Let $node \leftarrow k$

$node.left = BuildTree(i, k-1)$

$node.right = BuildTree(k+1, j)$

 return $node$

Time Complexity:

States of $c(i, j)$: $O(n^2)$.

Every state need $O(n)$ time to get root k .

Compute $S[j] - S[i-1]$: $O(1)$ time.

Overall Time: $O(n^3)$.

Proof of Correctness: Seen in part **Subproblem Transfer** and **Topological Order**.

Question 3

By the problem statement, our objective is to solve Knapsack Problem with Tree constraint. Assume *Root* the root of the Tree.

Subproblem Definition

Define $f[u, k]$: In a subtree rooted as node u , the maximum total value when we choose k nodes in this subtree. It is obvious that u must be chosen if $k \neq 0$ ($f[u, 1] = v_u$), and $f[u, 0] = 0$.

Objective changes to find $f[\text{root}, W]$.

Subproblem Transfer

Assume root u has children $\{c_1, c_2, \dots, c_m\}$. If $k \geq 1$, we must choose node u with value v_u . The left $k - 1$ nodes will be assigned to children. Let child c_1 choose n_1 nodes, c_2 choose n_2 nodes, ... Then $n_1 + n_2 + \dots + n_m = k - 1$ holds.

So the state transfer:

$$f[u, k] = \begin{cases} 0, & k = 0 \\ v_u + \max\{\sum_{i \in [1, m]} f[c_i, n_i]\} & k \geq 1, n_1 + \dots + n_m = k - 1 \end{cases}$$

Topological Order

If u is the leaf node, then $f[u, 0] = 0, f[u, 1] = v_u, f[u, k] = -\infty$ when $k \geq 2$.

From the Transfer, $f[u, k]$ depends on $f[c_j, n_j]$, where c_j is a child of u . So if we compute with the Tree's Post Order (Compute children first, compute parent last), then we are absolutely in topological order.

Algorithm01

Algorithm TreeKnapsack(*root*, *W*):

PostOrder the tree.

Initialize $f[u, 0] = 0, f[u, k] = -\infty$ for each u in PostOrder.

If u is a leaf: $f[u, 1] = v_u$

Else

For $k = 1 \rightarrow W$:

$$f[u, k] = v_u + \max\{f[c_1, n_1], f[c_2, n_2], \dots, f[c_m, n_m]\} \text{ for } \sum_1^m n_i = k - 1$$

Endfor

EndIf

return $\max_{0 \leq k \leq W} f[\text{Root}, k]$

Time Complexity

For each node u , we need to compute $(W + 1)$ $f[u, k]$ states.

Compute $f[u, k]$: we should assign $k - 1$ nodes to every child subtree. Every transfer needs $O(W)$ time to get the number of chosen children.

There are $n - 1$ parent-child edges.

$\implies$ Overall Time: $O(nW^2)$.

Proof of Correctness

We can know DP's transfer function and topological order are correct by the analysis above.

Improvement

In Algorithm01 above, we spend much time to assign capacity W to every children. We can come up with an idea that no longer enumerate capacity assignment for every children.

- **Use PreOrder:** PreOrder the Tree, give each node number $1, 2, \dots, n$ in PreOrder. Maintain an array $next[i]$, to record the next location after Subtree(i). If not choose node i , all i 's descendants will not be chosen. So skip to $next[i]$ directly.
- **Subproblem Definition:** Consider remaining capacity w , where $0 \leq w \leq W$. Define $f[i, w]$: the maximum value we can get, considering from node i and remaining capacity w .
- **Subproblem Transfer:** for the i -th node in PreOrder, only two cases below are possible:
 - Case 1.* not choose $i \rightarrow$ then the whole subtree(i) should not be chosen.
 $f[i, w] = f[next[i], w]$.
 - Case 2.* choose $i \rightarrow$ spend one capacity, get value v_i . i 's children is available to be chosen.
 $f[i, w] = v_i + f[i + 1, w - 1]$.
 So we have: $f[i, w] = \max\{f[next[i], w], v_i + f[i + 1, w - 1]\}$
- **Topological Order:** $f[i, 0] = 0, f[n + 1, w] = 0$;
 State $f[i, w]$ depends on $f[next[i], w]$ and $f[i + 1, w - 1]$. Since $next[i] > i$, $f[i, w]$ only depends on the state with larger index.
 We can compute with the order $i = n, n - 1, \dots, 1$. It is the correct topological order.

Algorithm02

Algorithm OptTreeKnapsack(root, W):

```

    Do PreOrder from Root; Rename node as 1, 2, ..., n by PreOrder.
    Compute next[i] for each  $i$ 
    Initialize  $f[n + 1, w] = 0$  for  $w = 0 \rightarrow W$ 
    For  $i = n \rightarrow 1$  :
         $f[i, 0] = 0$ 
        For  $w = 1 \rightarrow W$  :
             $f[i, w] = \max\{f[next[i], w], v_i + f[i + 1, w - 1]\}$ 
        EndFor
    EndFor
    return  $f[1, W]$ 

```

Time Complexity

PreOrder each node once: $O(n)$ time.

Compute every Transfer: $O(1)$ time.

Update remaining capacity w : from W to 0, W rounds.

$\implies$ Overall Time: $O(nW)$.

Proof of Correctness

From analysis above, we can know that the definition with PreOrder, Transfer function are correct; DP algorithm compute with correct topological order.

Question 4

I spend 4-5 hours to finish assignment4 this time.

Difficulty Score: 5

I did the assignment by myself, with some help of ideas by GPT.